

Библиотека NumPy

Библиотека NumPy лежит в основе стека машинного обучения на Python и позволяет эффективно работать со структурами данных, часто используемыми в машинном самообучении: векторами, матрицами и тензорами.

```
In [1]: import numpy as np
```

1.1 Создание вектора

```
In [4]: # Создать вектор как строку  
vector_row = np.array([1, 2, 3])
```

```
In [6]: vector_row
```

```
Out[6]: array([1, 2, 3])
```

```
In [8]: print(vector_row)
```

```
[1 2 3]
```

```
In [10]: # Создать вектор как столбец  
vector_column = np.array([[1], [2], [3]])
```

```
In [12]: vector_column
```

```
Out[12]: array([[1],  
                [2],  
                [3]])
```

```
In [14]: # Формирование вектора  
np.arange(0,10,2)
```

```
Out[14]: array([0, 2, 4, 6, 8])
```

```
In [16]: # Вектор из нулевых элементов  
np.zeros(9)
```

```
Out[16]: array([0., 0., 0., 0., 0., 0., 0., 0., 0.])
```

```
In [18]: # Единичный вектор  
np.ones(5)
```

```
Out[18]: array([1., 1., 1., 1., 1.])
```

```
In [20]: # Равномернораспределенный на отрезке вектор  
np.linspace(0, 10, 5)
```

```
Out[20]: array([ 0. ,  2.5,  5. ,  7.5, 10. ])
```

```
In [22]: np.linspace(0, 5, 5)
```

```
Out[22]: array([0. , 1.25, 2.5 , 3.75, 5. ])
```

```
In [24]: np.linspace(0, 5, 10)
```

```
Out[24]: array([0.          , 0.55555556, 1.11111111, 1.66666667, 2.22222222,  
                2.77777778, 3.33333333, 3.88888889, 4.44444444, 5.          ])
```

1.2 Создание массива

```
In [27]: # Создать матрицу  
matrix = np.array([[1, 2], [1, 2 ], [1, 2]])
```

```
In [29]: matrix
```

```
Out[29]: array([[1, 2],  
               [1, 2],  
               [1, 2]])
```

```
In [31]: # Массив из нулевых элементов  
np.zeros((2,3))
```

```
Out[31]: array([[0., 0., 0.],  
               [0., 0., 0.]])
```

```
In [33]: # Единичный массив  
np.ones((3,2))
```

```
Out[33]: array([[1., 1.],  
               [1., 1.],  
               [1., 1.]])
```

```
In [35]: # Единичная диагональная матрица  
np.eye(3)
```

```
Out[35]: array([[1., 0., 0.],  
               [0., 1., 0.],  
               [0., 0., 1.]])
```

1.3. Генерирование случайных значений

```
In [38]: # Задать начальное значение для генератора псевдослучайных чисел  
np.random.seed(0)
```

```
In [40]: # Сгенерировать три случайных вещественных числа между 0.0 и 1.0  
np.random.random(4)
```

```
Out[40]: array([0.5488135 , 0.71518937, 0.60276338, 0.54488318])
```

```
In [42]: np.random.rand(3, 4)
```

```
Out[42]: array([[0.4236548 , 0.64589411, 0.43758721, 0.891773  ],  
               [0.96366276, 0.38344152, 0.79172504, 0.52889492],  
               [0.56804456, 0.92559664, 0.07103606, 0.0871293 ]])
```

```
In [44]: # Сгенерировать три случайных целых числа между 1 и 10  
np.random.randint(0, 11, 3)
```

```
Out[44]: array([3, 0, 3])
```

```
In [46]: # Сгенерировать нормальнораспределенный массив  
np.random.randn(7)
```

```
Out[46]: array([-0.07770457,  1.08963016,  0.09654267,  1.41866711,  1.16827314,  
               0.94718595,  1.08548703])
```

```
In [48]: np.random.randn(3, 4)
```

```
Out[48]: array([[ 2.38222445, -0.40602374,  0.26644534, -1.35571372],  
               [-0.11410253, -0.84423086,  0.70564081, -0.39878617],  
               [-0.82719653, -0.4157447 , -0.52451219,  0.81310127]])
```

```
In [50]: # Извлечь три числа из нормального распределения со средним, равным 0.0,  
# и стандартным отклонением, равным 1.0  
np.random.normal(0.0, 1.0, 3)
```

```
Out[50]: array([-0.22925063,  2.16171737, -0.95693143])
```

1.4. Выбор элементов

```
In [53]: # Создать вектор-строку  
vector = np.array([1, 2, 3, 4, 5, 6])  
  
# Создать матрицу  
matrix = np.array([[1, 2, 3],  
                   [4, 5, 6],  
                   [7, 8, 9]])
```

```
In [55]: # Выбрать третий элемент вектора  
vector[2]
```

```
Out[55]: 3
```

```
In [57]: # Выбрать элемент на второй строке, во втором столбце  
matrix[1, 1]
```

```
Out[57]: 5
```

```
In [59]: # Выбрать все элементы вектора  
vector[:]
```

```
Out[59]: array([1, 2, 3, 4, 5, 6])
```

```
In [61]: # Выбрать все вплоть до третьего элемента включительно  
vector[:3]
```

```
Out[61]: array([1, 2, 3])
```

```
In [63]: # Выбрать все после третьего элемента  
vector[3:]
```

```
Out[63]: array([4, 5, 6])
```

```
In [65]: # Выбрать последний элемент  
vector[-1]
```

```
Out[65]: 6
```

```
In [67]: # Выбрать первые две строки и все столбцы матрицы  
matrix[:2,:]
```

```
Out[67]: array([[1, 2, 3],  
               [4, 5, 6]])
```

```
In [69]: # Выбрать все строки и второй столбец  
matrix[:,1:2]
```

```
Out[69]: array([[2],  
               [5],  
               [8]])
```

1.5. Описание матрицы

```
In [72]: # Создать матрицу
matrix = np.array([[1, 2, 3, 4],
                   [5, 6, 7, 8],
                   [9, 10, 11, 12]])
```

```
In [74]: matrix
```

```
Out[74]: array([[ 1,  2,  3,  4],
                [ 5,  6,  7,  8],
                [ 9, 10, 11, 12]])
```

```
In [76]: # Взглянуть на количество строк и столбцов
matrix.shape
```

```
Out[76]: (3, 4)
```

```
In [78]: # Взглянуть на количество элементов (строки * столбцы)
matrix.size
```

```
Out[78]: 12
```

```
In [80]: # Взглянуть на количество размерностей
matrix.ndim
```

```
Out[80]: 2
```

1.6. Операции над массивами

```
In [83]: # Создать матрицу
matrix = np.array([[1, 2, 3, 4],
                   [5, 6, 7, 8],
                   [9, 10, 11, 12]])
```

1.6.1 Нахождение максимального и минимального элементов

```
In [86]: # Вернуть максимальный элемент
np.max(matrix)
```

```
Out[86]: 12
```

```
In [88]: # Вернуть минимальный элемент
matrix.min()
```

```
Out[88]: 1
```

```
In [90]: # Вернуть индекс максимального элемента
matrix.argmax()
```

```
Out[90]: 11
```

```
In [92]: # Найти максимальный элемент в каждом столбце
np.max(matrix, axis=0)
```

```
Out[92]: array([ 9, 10, 11, 12])
```

```
In [94]: # Найти максимальный элемент в каждой строке
np.max(matrix, axis=1)
```

```
Out[94]: array([ 4,  8, 12])
```

1.6.2 Вычисление среднего значения, дисперсии и стандартного отклонения

```
In [97]: # Вернуть среднее значение
np.mean(matrix)
```

```
Out[97]: 6.5
```

```
In [99]: # Вернуть дисперсию
np.var(matrix)
```

```
Out[99]: 11.916666666666666
```

```
In [101... # Вернуть стандартное отклонение
np.std(matrix)
```

```
Out[101... 3.452052529534663
```

```
In [103... # Найти среднее значение в каждом столбце
np.mean(matrix, axis=0)
```

```
Out[103... array([5., 6., 7., 8.])
```

1.6.3 Реформирование массивов

Метод reshape позволяет реструктурировать массив так, что мы сохраняем те же самые данные и при этом организуем их как другое количество строк и столбцов. Единственное требование состоит в том, чтобы формы исходной и новой матриц содержали одинаковое количество элементов (т.е. матрицы имели одинаковый размер).

```
In [107... rand_arr = np.random.randint(0,10,20)
```

```
In [109... rand_arr
```

```
Out[109... array([1, 1, 7, 9, 9, 3, 6, 7, 2, 0, 3, 5, 9, 4, 4, 6, 4, 4, 3, 4])
```

```
In [111... matrix = rand_arr.reshape(4,5)
```

```
In [113... matrix
```

```
Out[113... array([[1, 1, 7, 9, 9],
        [3, 6, 7, 2, 0],
        [3, 5, 9, 4, 4],
        [6, 4, 4, 3, 4]])
```

```
In [115... matrix.reshape(5, 4)
```

```
Out[115... array([[1, 1, 7, 9],
        [9, 3, 6, 7],
        [2, 0, 3, 5],
        [9, 4, 4, 6],
        [4, 4, 3, 4]])
```

1.6.4 Сглаживание матрицы

Метод flatten представляет собой простой метод преобразования матрицы в одномерный массив.

```
In [119... # Сгладить матрицу
matrix.flatten()
```

```
Out[119... array([1, 1, 7, 9, 9, 3, 6, 7, 2, 0, 3, 5, 9, 4, 4, 6, 4, 4, 3, 4])
```

```
In [121... matrix.reshape(1, -1)
```

```
Out[121...] array([[1, 1, 7, 9, 9, 3, 6, 7, 2, 0, 3, 5, 9, 4, 4, 6, 4, 4, 3, 4]])
```

1.6.5 Транспонирование

Транспонирование — это универсальная линейно-алгебраическая операция, в которой индексы столбцов и строк каждого элемента меняются местами

```
In [125...] # Транспонировать матрицу
matrix.T
```

```
Out[125...] array([[1, 3, 3, 6],
                [1, 6, 5, 4],
                [7, 7, 9, 4],
                [9, 2, 4, 3],
                [9, 0, 4, 4]])
```

1.6.6 Определитель матрицы

```
In [128...] matrix = np.array([[1, 2, 3],
                              [2, 4, 6],
                              [3, 8, 9]])
```

```
In [130...] # Вернуть определитель матрицы
np.linalg.det(matrix)
```

```
Out[130...] 0.0
```

```
In [132...] # Вернуть диагональные элементы
matrix.diagonal()
```

```
Out[132...] array([1, 4, 9])
```

```
In [134...] # Вынуть след матрицы - сумма элементов главной диагонали
matrix.trace()
```

```
Out[134...] 14
```

```
In [136...] # Вернуть диагональ и сумму ее элементов
sum(matrix.diagonal())
```

```
Out[136...] 14
```

1.6.7 Вычисление скалярных произведений

```
In [139...] # Создать два вектора
vector_a = np.array([1,2,3])
vector_b = np.array([4,5,6])
```

```
In [141...] # Вычислить скалярное произведение
np.dot(vector_a, vector_b)
```

```
Out[141...] 32
```

1.6.8 Арифметические операции над матрицами

```
In [144...] # Создать матрицу
matrix_a = np.array([[1, 1, 1],
                    [1, 1, 1],
                    [1, 1, 2]])
```

```
In [146...] # Создать матрицу
matrix_b = np.array([[1, 3, 1],
```

```
[1, 3, 1],  
[1, 3, 8]])
```

```
In [148... # Сложить две матрицы  
np.add(matrix_a, matrix_b)
```

```
Out[148... array([[ 2,  4,  2],  
        [ 2,  4,  2],  
        [ 2,  4, 10]])
```

```
In [150... # Вычесть из одной матрицы другую  
np.subtract(matrix_a, matrix_b)
```

```
Out[150... array([[ 0, -2,  0],  
        [ 0, -2,  0],  
        [ 0, -2, -6]])
```

```
In [152... # Сложить две матрицы  
matrix_a + matrix_b
```

```
Out[152... array([[ 2,  4,  2],  
        [ 2,  4,  2],  
        [ 2,  4, 10]])
```

```
In [154... # Вычесть из одной матрицы другую  
matrix_a - matrix_b
```

```
Out[154... array([[ 0, -2,  0],  
        [ 0, -2,  0],  
        [ 0, -2, -6]])
```

```
In [156... # Перемножить две матрицы  
np.dot(matrix_a, matrix_b)
```

```
Out[156... array([[ 3,  9, 10],  
        [ 3,  9, 10],  
        [ 4, 12, 18]])
```

```
In [158... # Перемножить две матрицы  
matrix_a @ matrix_b
```

```
Out[158... array([[ 3,  9, 10],  
        [ 3,  9, 10],  
        [ 4, 12, 18]])
```

```
In [160... # Создать матрицу  
matrix = np.array([[1, 4],  
                  [2, 5]])
```

```
In [162... # Вычислить обратную матрицу  
np.linalg.inv(matrix)
```

```
Out[162... array([[-1.66666667,  1.33333333],  
        [ 0.66666667, -0.33333333]])
```

```
In [164... np.sqrt(matrix)
```

```
Out[164... array([[1.         , 2.         ],  
        [1.41421356, 2.23606798]])
```

```
In [ ]:
```

```
In [ ]:
```